

An
Industrial Training
Report On



**Application Development using Python of JNARDDC,
Wadi, Nagpur**

Submitted by:

**Ms. Shruti N. Pethe
Mr. Om P. Mandwade
Mr. Ritesh G. Nayase**

DEPARTMENT OF INFORMATION TECHNOLOGY



**Government Polytechnic, Nagpur
(An Autonomous Institute of Government of Maharashtra)
Near Mangalwari Bazar, Sadar, Nagpur-440001**



**Jawaharlal Nehru Aluminium Research
Development and Design Centre**
(An Autonomous Body Under Ministry of Mines, Government of India.)

DECLARATION

During my internship in Jawaharlal Nehru Aluminium Research Development and Design Centre (JNARDDC), Nagpur and in preparation of this report I realized that it is the joint venture guidance, assistance and cooperation. So, it would have not been completed without a declaration and help received. It is a matter of great privilege to express my deep sense of gratitude towards my guide Mr. V.K. JHA During my internship in Jawaharlal Nehru Aluminium Research Development and Design Centre (JNARDDC), Nagpur. For having this guidance, I am extremely thankful to him for constant motivation and inspiration extended throughout during internship work which has made it possible to complete the work in scheduled time. My sincere thanks to all the faculties.

Dr. Anupam Agnihotri
(Director, JNARDDC)

Mr. Manoj Nimje
(HOD)

Mr. V. K. Jha
(Senior Scientist, JNRDDC)

Index

Sr. No.	Title of Content	Page No.
1	Abstract	5
2	Project Genesis 2.1 Introduction 2.2 Problem statement 2.3 Problem solution 2.3.1 Edge detection 2.3.2 Anode number Tracking	6
3	Introduction 3.1 Overview of Project 3.2 History 3.3 Features of Python 3.4 Python and Internet 3.5 Python and World Wide Web 3.6 Python enabled browsers 3.7 Hardware and Software Requirements 3.8 Python Environment	9
4	GUI (Graphical User Interface) 4.1 Tkinter 4.2 PyQt / PySide 4.3 Kivy 4.4 wxPython 4.5 PyGTK / PyGObject 4.6 wxPython	12
5	MYSQL WORKBENCH 5.1 Database Design and Modeling 5.2 SQL Development 5.3 Database Administration 5.4 Collaboration and Teamwork 5.5 Cross-Platform Availability 5.6 Extensibility	15

6	Python Database Connectivity 6.1 Database Libraries and Modules 6.2 Database Connectivity Steps 6.3 Example Code (Using “mysql.connector”) 6.4 Additional Considerations	17
7	Image Recognition & Analysis in Python 7.1 Image Processing 7.2 Image analysis	20
8	Python Libraries and Frameworks 8.1 OpenCV (Open-Source Computer Vision Library) 8.2 Numpy 8.3 os 8.4 pandas 8.5 Pillow (Python Imaging Library) 8.6 Pylon 8.7 easyocr	22
9	Basler camera	25
10	Pylon viewer	26
11	Project Codes 11.1 Anode Tracking system [Source code] 11.2 Steps & Output 11.3 Anode Edge Detection System [Source code] 11.4 Steps & Output	27
12	Conclusion	50
13	Reference	51

1.Abstract

In this report, we designed and developed an application in Python for edge and number detection system. Unlike Java Swing, which can be complex, our Python application simplifies the process by leveraging Python's libraries for computer vision tasks. The application allows users to input data through a form, which is then processed for edge detection and number recognition. The detected information is stored in a database, facilitating easy retrieval and manipulation of records.

Using Python's standard libraries and frameworks such as OpenCV and NumPy, we created an intuitive interface for users to interact with the system. The primary objective of this project is to enable seamless interaction with computer vision algorithms, allowing users to detect edges and recognize numbers efficiently. Implemented within the Python programming language environment, this application offers a user-friendly approach to database interaction and image processing tasks.

Through this project, we demonstrate the potential of Python for developing sophisticated graphical applications, particularly in the domain of computer vision. By harnessing the power of Python's libraries and frameworks, we provide a seamless and intuitive solution for edge and number detection, enhancing accessibility and usability for users.

2.Project Genesis

2.1 Introduction:

Edge detection and number tracking are essential components of various computer vision applications, ranging from surveillance systems to autonomous vehicles. In this project, we aim to develop a Python application that combines these two functionalities to detect edges in images or video streams and track numbers as they move within the captured frames.

The project involves the implementation of advanced image processing techniques and computer vision algorithms to achieve accurate edge detection and number recognition. By integrating these components, the application will be capable of real-time analysis of visual data, enabling tasks such as object tracking and motion detection.

Through this project, we aim to demonstrate the potential of Python and its libraries for solving complex computer vision tasks. Additionally, the developed application can serve as a foundation for further research and development in areas such as surveillance, object tracking, and augmented reality.

2.2 Problem Statement:

Develop a Python application capable of performing edge detection and number tracking. The application should be able to identify edges in images or video streams and track numbers as they move within the captured frames. This project involves two main components: edge detection and number recognition.

2.3 Problem Solution:

2.3.1 Edge Detection:

1. Upload Image:

- Implement a feature in the application to allow users to upload an image from their local system or provide a URL to an image hosted online.

- Utilize libraries such as `cv2` in OpenCV or `Pillow` for image loading and manipulation.
- Ensure error handling for cases such as invalid file formats or inaccessible URLs.

2. Apply Filters

- Implement a series of filters to preprocess the uploaded image before edge detection.
- Preprocessing filters may include techniques such as:
 - Grayscale conversion: Convert the uploaded image to a single-channel grayscale image to simplify processing.
 - Gaussian blur: Apply Gaussian blur to reduce noise and smooth out the image.
 - Morphological operations: Perform operations like erosion and dilation to enhance edges and remove small imperfections.
 - Thresholding: Apply thresholding techniques to segment the image and separate regions of interest.
- Implement these filters using functions provided by OpenCV or custom filter implementations.

3. Edge Detection

- Apply edge detection algorithms to the preprocessed image to highlight regions with significant changes in intensity.
- Common edge detection algorithms include Canny edge detection and Sobel operator.
- Use appropriate parameters for these algorithms to achieve optimal edge detection results, considering factors such as image noise and edge continuity.

4. Download Result:

- Provide users with the option to download the processed image containing detected edges.
- Implement functionality to save the processed image to the local system with a user-specified filename and format.
- Ensure compatibility with common image formats such as JPEG or PNG.

2.3.2 Number Extraction

1. Image Processing and Number Extraction:

- Utilize OpenCV library to read and process images containing numerical characters.
- Implement preprocessing techniques such as resizing, grayscale conversion, and noise reduction to enhance the quality of the input images.
- Use techniques like thresholding, contour detection, and character segmentation to isolate individual numerical characters within the image.

2. Number Recognition and Extraction:

- Train machine learning models or use pre-trained models to recognize and extract numerical characters from the segmented regions.
- Implement algorithms to ensure accurate recognition of digits, considering variations in font styles, sizes, and orientations.
- Develop a system to handle cases of multiple digits within a single image and extract each digit separately.

3. Database Integration:

- Utilize a database management system, MySQL workbench) to store the extracted numbers.
- Design an appropriate database schema to accommodate the extracted numbers along with any additional metadata such as image filenames or timestamps.
- Implement functions to establish a connection with the database, insert the extracted numbers into the database, and handle any errors or exceptions during the database operations.

4. User Interface (Optional):

- Develop a user interface using libraries such as Tkinter to provide a graphical interface for users to interact with the application.
- Include functionalities such as image upload, processing, and database interaction within the user interface to streamline the user experience.

3. Introduction

3.1 Overview of Python

Python is a high-level, interpreted programming language known for its simplicity and readability. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and allows programmers to express concepts in fewer lines of code compared to other languages, making it an ideal choice for beginners and experienced developers alike.

3.2 History of Python

Python was conceived in the late 1980s by Guido van Rossum at Centrum Wiskunde & Informatica (CWI) in the Netherlands. It was designed as a successor to the ABC programming language. Python's first version, 0.9.0, was released in February 1991. Since then, Python has undergone numerous updates and revisions, with Python 2.x and Python 3.x being the major releases.

3.3 Features of Python

- Python boasts several features, including:
 - **Simple and easy to learn syntax:** Python's syntax is clean and easy to read, reducing the cost of program maintenance and development
 - **High-level language:** Python abstracts away low-level details, allowing programmers to focus on solving problems rather than managing memory allocation or other system-specific concerns.
 - **Interpreted:** Python code is executed line by line, making development and debugging faster.
 - **Dynamically typed:** Python variables don't need explicit declaration, allowing for more flexibility but requiring careful attention to data types.
 - **Rich standard library:** Python comes with a vast collection of libraries and modules that simplify tasks ranging from web development to data analysis.

3.4 Python and the Internet

- Python plays a significant role in various aspects of internet technology. It is used for applications.
 - **Networking:** Python's libraries such as Requests make it easy to interact with web services and APIs.
 - **Data scraping:** Libraries like BeautifulSoup and Scrapy are commonly used for extracting data from websites.

3.5 Python and the World Wide Web

- Python's versatility extends to the World Wide Web, where it is used for:
 - **Server-side scripting:** Python is often employed on the server-side to generate dynamic web content.
 - **Client-side scripting:** With tools like Brython, Python code can be executed directly in the browser.
 - Web crawling and scraping Python's libraries enable automated extraction of data from websites.

3.6 Python-enabled Browsers

While Python is typically not directly executed within browsers like JavaScript, there are projects like Brython that allow Python code to run in a web browser environment. This enables developers to write client-side web applications using Python instead of JavaScript.

3.7 Hardware and Software Requirements

Python is highly portable and can run on various hardware and software platforms. It is supported on major operating systems such as Windows, macOS, and Linux. Python can run on anything from personal computers to servers and even embedded devices. The hardware requirements for running Python are minimal, with most

modern computers capable of running Python without any issues.

3.8 Python Environment in Brief

- **Interpreter:** The Python interpreter executes Python code.
- **Standard library:** A vast collection of modules and packages that come with Python, providing functionality for tasks like file I/O, networking, and more.
- **Integrated Development Environment (IDE):** Tools like PyCharm, VS Code, and IDLE provide a development environment with features such as code highlighting, debugging, and management.
- **Package managers:** Tools like pip are used to install and manage additional Python packages and dependencies.
- **Virtual environments:** Virtual env or venv allows developers to create isolated Python environments for projects, managing dependencies separately from the system Python installation.

4.GUI Framework

4.1 Tkinter:

- Tkinter is the standard GUI toolkit included with Python.
- It provides a simple and easy-to-use interface for creating basic GUI applications.
- Tkinter is based on the Tk GUI toolkit, which is a cross-platform library.
- Despite its simplicity, Tkinter is powerful enough to create a wide range of GUI applications, from simple tools to complex desktop applications.

4.2 PyQt / PySide:

- PyQt and PySide are Python bindings for the Qt framework.
- Qt is a powerful and versatile C++ framework for building cross-platform applications with native-looking GUIs.
- PyQt and PySide provide Python developers access to Qt's extensive features and widgets for creating sophisticated GUI applications.
- PyQt is developed by Riverbank Computing, while PySide is maintained by the Qt Company.

4.3 Kivy:

- Kivy is an open-source Python framework for developing multitouch applications.
- It is designed for building cross-platform applications that run on Windows, macOS, Linux, iOS, Android, and Raspberry Pi.
- Kivy uses a natural user interface (NUI) approach and provides a rich set of customizable widgets for creating interactive and dynamic user interfaces.
- kiosk applications.

4.4 wxPython:

- wxPython is a Python wrapper for the wxWidgets C++ library.
- wxWidgets is a mature and feature-rich cross-platform GUI toolkit.
- wxPython allows Python developers to create native-looking applications with a wide range of widgets and controls.
- It supports various platforms, including Windows, macOS, and Linux, making it suitable for developing cross-platform desktop applications.

4.5 PyGTK / PyGObject:

- PyGTK is a set of Python bindings for the GTK+ toolkit.
- GTK+ is a popular open-source toolkit for creating graphical user interfaces.
- PyGTK provides Python developers access to GTK+'s widgets and functionality for building desktop applications.
- PyGObject is the newer version of PyGTK, which provides Object introspection-based bindings for GTK+ and other GNOME libraries.

4.6 wxPython:

- wxPython is a Python wrapper for the wxWidgets C++ library.
- wxWidgets is a mature and feature-rich cross-platform GUI toolkit.
- wxPython allows Python developers to create native-looking applications with a wide range of widgets and controls.
- It supports various platforms, including Windows, macOS, and Linux, making it suitable for developing cross-platform desktop applications.

These are just a few of the many options available for creating GUI applications in Python. The choice of library or framework depends on factors such as project requirements, platform support, licensing, and personal Preference.

5. MySQL Workbench

MySQL Workbench is a visual database design and modeling tool, SQL development environment, and administration tool for MySQL databases. It's developed by Oracle Corporation and is available for free as part of the MySQL software stack. Here is a breakdown of its features and functionalities:

5.1 Database Design and Modeling:

- **Entity-Relationship Diagrams (ERD):** Allows users to visually design and model databases using ERDs, representing tables, columns, relationships, and constraints.
- **Forward and Reverse Engineering:** Supports forward engineering to generate SQL scripts from ERD models and reverse engineering to create models from existing databases.
- **Schema Synchronization:** Provides tools to compare and synchronize model schemas with actual databases, ensuring consistency between design and implementation.
- **Database Documentation:** Enables the generation of detailed documentation for database schemas and models.

5.2 SQL Development:

- **SQL Editor:** Offers a robust SQL editor with syntax highlighting, code completion, and query execution capabilities.
- **Query History:** Keeps track of executed queries for easy reference and reuse.
- **Visual Query Build:** Allows users to construct SQL queries visually using a drag-and-drop interface, especially helpful for those unfamiliar with SQL syntax.
- **Code Snippets:** Provides a library of pre-defined code snippets for commonly used SQL commands and functions, speeding up development.

5.3 Database Administration:

- **Server Administration:** Allows administrators to manage MySQL server instances, including server configuration, monitoring, and status checks.
- **User Management:** Facilitates the creation, modification, and deletion of user accounts and privileges for accessing the database.
- **Backup and Restore:** Provides tools for performing database backups and restores, including full, incremental, and partial backups.
- **Performance Tuning:** Includes features for analyzing database performance, optimizing queries, and monitoring server health.

5.4 Collaboration and Teamwork:

- **Model Sharing:** Supports collaboration by allowing users to share database models with team members or collaborators.
- **Version Control Integration:** Integrates with version control systems like Git, enabling versioning and collaborative development of database models and scripts.

5.5 Cross-Platform Availability:

MySQL Workbench is available for Windows, macOS, and Linux, ensuring compatibility across different operating systems.

5.6 Extensibility:

Supports extensibility through plugins, allowing users to add custom functionality or integrate with third-party tools and services. MySQL Workbench is a comprehensive tool for database professionals, offering a user-friendly interface and a wide range of features for database design, development, and administration **tasks**.

6. Python Database Connectivity

Python offers various libraries and modules for database connectivity, allowing developers to interact with different database management systems (DBMS) from their Python applications. Here is some information about database connectivity in Python:

6.1 Database Libraries and Modules:

- **sqlite3:** Included in Python's standard library, `sqlite3` provides a lightweight and self-contained database engine for working with SQLite databases.
- **psycopg2:** A popular PostgreSQL adapter for Python, enabling connectivity and interaction with PostgreSQL databases.
- **Mysql-connector-python:** An official MySQL driver for Python provided by Oracle, offering connectivity to MySQL databases.
- **Pyodbc:** A Python module for interacting with ODBC (Open Database Connectivity) databases, providing a common interface for various database systems.
- **Pymysql:** A pure-Python MySQL client library that implements the MySQL protocol, offering an alternative to `mysql-connector-python`.
- **sqlalchemy:** A comprehensive SQL toolkit and Object-Relational Mapping (ORM) library that supports various DBMS including SQLite, PostgreSQL, MySQL, and more.

6.2 Database Connectivity Steps:

- **Importing the Required Module:** Import the appropriate module or library for the target database system.
- **Establishing Connection:** Use the module's functions or methods to establish a connection to the database server, providing necessary connection parameters like host, port, username, password, and database name.
- **Executing Queries:** Once connected, execute SQL queries or commands using methods provided by the module, such as `execute()` or `executemany()`.
- **Fetching Data:** Retrieve query results using methods like `fetchone()`, `fetchall()`, or iterators, depending on the expected result set size.

- **Handling Transactions:** Manage transactions by committing or rolling back changes using methods like `commit()` and `rollback()`.
- **Closing Connection:** Close the database connection when done to release resources using the `close()` method.

6.3 Example code (Using “mysql.connector”)

```
import pymysql

# Establish connection
conn = pymysql.connect(
    host='localhost',
    user='your_username',
    password='your_password',
    database='your_database_name'
)
# Create cursor object
cursor = conn.cursor()

# Execute SQL query to create table if not exists
cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(255),
        email VARCHAR(255)
    )
""")

# Insert data
cursor.execute('INSERT INTO users (name, email) VALUES (%s, %s)', ('John Doe',
'john@example.com'))

# Fetch data
cursor.execute('SELECT * FROM users')
rows = cursor.fetchall()
for row in rows:
    print(row)

# Commit changes and close connection
conn.commit()
conn.close()
```

6.4 Additional Considerations:

- **Error Handling:** Implement proper error handling to manage exceptions during database operations
- **Security:** Use parameterized queries or prepared statements to prevent SQL injection attacks.
- **Data Types:** Be aware of data types and data integrity constraints supported by the target database system.
- **Performance:** Optimize database interactions for better performance, considering factors like connection pooling and query optimization.

By leveraging these libraries and following best practices, Python developers can effectively connect to and interact with various database systems from their applications.

7. Image Processing & Analysis in Python

7.1 Image Processing:

- **Description:** Image recognition, also known as computer vision, is the process of identifying and categorizing objects or patterns within digital images or videos.
- **Methodology:** It involves analyzing the visual features of an image to classify or recognize objects or scenes present in the image.
- **Techniques:** Image recognition techniques often utilize machine learning algorithms, particularly deep learning models such as convolutional neural networks (CNNs), to automatically learn and extract relevant features from images.
- **Applications:** Image recognition has various applications, including facial recognition, object detection, handwriting recognition, optical character recognition (OCR), medical image analysis, and autonomous vehicles.

7.2 Image Analysis:

- **Description:** Image analysis is the process of extracting meaningful information from images, typically involving preprocessing, segmentation, feature extraction, and interpretation.
- **Preprocessing:** Involves techniques such as noise reduction, image enhancement, and normalization to improve the quality of images for analysis.
- **Segmentation:** Refers to dividing an image into meaningful regions or segments based on characteristics such as color, texture, or intensity.
- **Feature Extraction:** Involves extracting relevant features or attributes from segmented regions to represent the content of the image.
- **Interpretation:** Refers to the analysis and interpretation of extracted features to derive insights or make decisions.

- **Applications:** Image analysis finds applications in various fields such as medical imaging (e.g., tumor detection), remote sensing, agriculture (e.g., crop monitoring), surveillance, quality control, and scientific research.

7.3 Python Libraries and Frameworks:

- Python provides several libraries and frameworks for image recognition and analysis, including OpenCV, TensorFlow, Keras, PyTorch, scikit-image, and Pillow. These libraries offer a wide range of functionalities for tasks such as image preprocessing, feature extraction, object detection, classification, segmentation, and visualization.
- Developers can leverage these libraries to build custom image recognition and analysis pipelines tailored to their specific needs and applications.

In summary, image recognition involves identifying objects or patterns within images using machine learning techniques, while image analysis focuses on extracting meaningful information from images for interpretation and decision-making. Python provides a rich ecosystem of libraries and frameworks for implementing these tasks efficiently and effectively.

8. Image Processing and image analysis in Python involve the use of various libraries and frameworks to process and understand images.

8.1 OpenCV (Open Source Computer Vision Library):

- **Description:** OpenCV is a widely-used open-source library for computer vision and image processing tasks.
- **Features:** It offers a vast array of functionalities for tasks such as image manipulation, feature detection, object recognition, and more.
- **Usage:** OpenCV can be used for tasks like face detection, object tracking, image stitching, and augmented reality applications.

8.2 NumPy:

- **Description:** NumPy is a Python library designed for numerical computing, well-suited for handling large arrays of numeric data.
- **Features:** NumPy's ndarray enables efficient storage and manipulation of image data. Vectorized operations allow for fast element-wise operations on image arrays.
- **Usage:** Supports various image formats for loading and saving using libraries like PIL or OpenCV.

8.3 os:

- **Description:** The os module provides a portable way of interacting with the operating system, allowing Python programs to perform various operating system-related tasks such as file and directory manipulation, environment variables, and process management.
- **Usage:** It is commonly used for tasks such as file operations (e.g., reading, writing, renaming, deleting files), directory operations and accessing environment variables.
- **Features:** The os module offers functions for path manipulation, file and directory operations, process management, and environment variables handling.

8.4 pandas

- **Description:** Pandas is a powerful data manipulation and analysis library for Python, built on top of NumPy. It provides data structures and functions for working with structured data, such as data frames, series, and time series, making it a popular choice for data cleaning, exploration, and transformation tasks.
- **Usage:** Pandas is widely used for tasks such as data loading, cleaning, filtering, aggregation, and visualization. It is commonly used in data analysis, machine learning, and statistical modeling workflows.
- **Features:** Key features of pandas include data structures like DataFrame and Series, powerful indexing and slicing capabilities, data alignment and reshaping functions.

8.6 Pillow (Python Imaging Library):

- **Description:** Pillow is a fork of the Python Imaging Library (PIL) and provides a simple and powerful library for opening, manipulating, and saving many different image file formats.
- **Features:** It supports basic image processing operations such as resizing, cropping, rotating, and filtering.
- **Usage:** Pillow is commonly used for tasks like image loading, preprocessing, and saving in various image formats.

8.7 Pylon

- **Description:** Pypylon is a Python wrapper for the Basler pylon Camera Software Suite, allowing users to control Basler cameras from within Python scripts. It provides access to camera settings, image acquisition, and camera control functionalities.
- **Usage:** Pypylon is used for tasks such as configuring camera settings (e.g., exposure, gain, frame rate), capturing images and video streams, performing image processing on acquired images, and integrating Basler cameras into computer vision applications.
- **Features:** Pypylon offers a simple and intuitive interface for interacting with Basler cameras, support for camera discovery and connection management, access to camera features and parameters, and compatibility with various camera models and interfaces.

8.8 EasyOCR

- **Description:** EasyOCR is a Python library for optical character recognition (OCR), capable of extracting text from images with ease. It provides a simple interface for performing text detection and recognition tasks on images containing printed or handwritten text.
- **Usage:** EasyOCR is commonly used for tasks such as extracting text from images, processing scanned documents, automating data entry from images, and building text recognition systems.
- **Features:** EasyOCR offers pre-trained models for multiple languages, support for detecting and recognizing text in various fonts and styles, configurable accuracy and speed settings, and integration with other image processing libraries like OpenCV.

These are some of the main libraries and frameworks used for image recognition and image analysis tasks in Python. Depending on the specific requirements and complexity of the task, developers can choose the most suitable tool for their projects.

9. Basler Camera:

Basler AG is a leading manufacturer of industrial and machine vision cameras, offering a wide range of high-quality cameras for various applications.

➤ **Features:**

- **High Resolution:** Basler cameras are available in various resolutions, from standard definition to high-definition and beyond, to suit different imaging requirements.
- **High Speed:** Basler cameras offer high-speed imaging capabilities, enabling rapid image capture and processing for applications requiring fast-moving objects or processes.
- **Image Quality:** Basler cameras deliver excellent image quality with low noise, high dynamic range and accurate color reproduction, ensuring reliable and precise image analysis.
- **Interface Options:** Basler cameras support industry-standard interfaces such as GigE Vision, USB3 Vision, and Camera Link, providing flexibility and compatibility with different systems and devices.

- ### ➤ **Applications:**
- Basler cameras are used in a wide range of industries and applications, including factory automation, robotics, automotive inspection, medical imaging, surveillance, and scientific research.

10.Pylon Viewer:

- **Description:** Pylon Viewer is a lightweight and user-friendly software tool included in the Pylon software suite, designed for quick and easy camera evaluation, testing, and troubleshooting.
- **Features:**
 - **Live Image Preview:** Pylon Viewer provides a real-time live preview of the image captured by the connected Basler camera, allowing users to visualize the camera's output.
 - **Camera Configuration:** It offers basic camera configuration options for adjusting settings such as exposure time, gain, white balance, and image format.
 - **Image Capture:** Pylon Viewer allows users to capture still images and video sequences from the connected Basler camera for further analysis or documentation.
 - **Camera Information:** It provides detailed information about the connected Basler camera, including model name, serial number, firmware version, and supported features.
- **Usage:** Pylon Viewer is commonly used by developers, engineers, and technicians during the initial setup, testing, and evaluation of Basler cameras to ensure proper functionality and performance.
- **Compatibility:** Pylon Viewer is compatible with Basler's entire range of industrial and machine vision cameras, offering a convenient and intuitive interface for camera evaluation and testing.

11. Programs code

11.1 Anode Tracking system [Source code]

```
import cv2
import numpy as np
import tkinter as tk
from tkinter import filedialog, ttk, messagebox
from PIL import Image, ImageTk
import os

file_path = "" # Global variable to store the file path

def display_imageTk (image, container):
    try:
        # Clear existing image label if it exists
        for widget in container.winfo_children():
            if isinstance(widget, tk.Label):
                widget.destroy()

        image_pil = Image.fromarray(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
        container_width = container.winfo_width()
        container_height = container.winfo_height()

        # Resize the image to fit within the container

        image_pil.thumbnail((container_width, container_height), Image.ANTIALIAS)

        photo = ImageTk.PhotoImage(image=image_pil)
        label = tk.Label(container, image=photo)
        label.image = photo
        label.pack(side=tk.TOP, padx=10, pady=10)
        container.photo = photo
    except Exception as e:
        messagebox.showerror("Error", f"Error occurred while displaying image: {e}")

def apply_filters_and_display(root):
    global file_path # Declare file_path as global
    try:
        file_path = filedialog.askopenfilename()
        if file_path:
            original_image = cv2.imread(file_path)
```

```

gray_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2GRAY)
smoothed_image = cv2.GaussianBlur(gray_image, (0, 0), 5)
_, binary_image = cv2.threshold(smoothed_image, 127, 255,
cv2.THRESH_BINARY)
num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    binary_image, connectivity=8
)
for i in range(1, num_labels):
    if stats[i, cv2.CC_STAT_AREA] < 30000:
        binary_image[labels == i] = 0
transposed_image = cv2.transpose(binary_image)
(
    num_labels_transposed,
    labels_transposed,
    stats_transposed,
    centroids_transposed,
) = cv2.connectedComponentsWithStats(transposed_image, connectivity=8)
for i in range(1, num_labels_transposed):
    if stats_transposed[i, cv2.CC_STAT_AREA] < 2000:
        transposed_image[labels_transposed == i] = 0
final_image = cv2.transpose(transposed_image)
edges = cv2.Canny(final_image, 100, 200)
kernel = np.ones((5, 5), np.uint8)
dilated_image = cv2.dilate(edges, kernel, iterations=1)

```

Update images in containers

```

display_image_tk(original_image, container1)
display_image_tk(gray_image, container2)
display_image_tk(smoothed_image, container3)
display_image_tk(binary_image, container4)
display_image_tk(final_image, container5)
display_image_tk(edges, container6)
display_image_tk(dilated_image, container7)

```

Store images for downloading global images_for_download

```

images_for_download = {
    "Original_Image": original_image,
    "Grayscale_Image": gray_image,
    "Smoothed_Image": smoothed_image,
    "Binary_Image": binary_image,
    "Final_Image": final_image,
    "Edged_Image": edges,
    "Dilated_Image": dilated_image,
}

```

```

except Exception as e:
    messagebox.showerror(
        "Error", f"Error occurred while applying filters and displaying images: {e}"
    )

def download_image(image_name, image, suffix):
    try:
        image_pil = Image.fromarray(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
        image_path = os.path.join(
            os.path.expanduser("~"), "Downloads", f"{image_name}_{suffix}.png"
        )
        image_pil.save(image_path)
    except Exception as e:
        messagebox.showerror("Error", f"Error occurred while downloading image: {e}")

def download_all_images():
    global file_path # Access the global file_path
    if "images_for_download" in globals():
        try:
            file_name = os.path.basename(file_path).split(".")[
                0
            ] # Extracting the uploaded file name
            for image_name, image in images_for_download.items():
                image_type = image_name.split("_")[0] # Extracting the image type
                download_image(
                    f"{file_name}{image_type}{('_')[-1]}",
                    image,
                    image_name.split("_")[-1],
                ) # Modified file name

            messagebox.showinfo("Download Successful", "All images saved
successfully!")
        except Exception as e:
            messagebox.showerror(
                "Error", f"Error occurred while downloading images: {e}"
            )
    else:
        messagebox.showwarning("No Images", "No images available for download.")

root = tk.Tk()
root.title("Edge Detector")
root.geometry("1920x1080")
root.config(bg="#FBEEC1")

```

```

header_label = tk.Label(
    root,
    text="Jawaharlal Nehru Aluminum Research Development and Design Centre,
Nagpur",
    font=("Times New Roman", 30, "bold"), # Larger, bold font
    bg="#0077cc", # Blue background
    fg="white", # White text
)
header_label.pack(fill=tk.X)

upload_button = tk.Button(
    root,
    text="Upload and Apply Filters",
    command=lambda: apply_filters_and_display(root),
    bg="#0077cc",
    fg="white",
    font=("Times New Roman", 14, "bold"),
)
upload_button.pack(
    side=tk.TOP, pady=(20, 30)
) # Increase top padding to 20 and bottom padding to 30

download_button = tk.Button(
    root,
    text="Download All",
    command=download_all_images,
    bg="#0077cc",
    fg="white",
    font=("Times New Roman", 12, "bold"),
)
download_button.pack(side=tk.TOP, pady=(0, 10)) # Add padding

```

Create a frame with a custom background color for the central area

```

center_frame = tk.Frame(root, bg="#FBEEC1")
center_frame.pack(fill=tk.BOTH, expand=True)
# Create canvas and scrollbar inside the central frame
canvas = tk.Canvas(center_frame, bg="#FBEEC1")
canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

scrollbar = ttk.Scrollbar(center_frame, orient=tk.VERTICAL,
command=canvas.yview)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

canvas.configure(yscrollcommand=scrollbar.set)

```

```

canvas.bind("<Configure>", lambda e:
canvas.configure(scrollregion=canvas.bbox("all")))
canvas.bind_all(
"<MouseWheel>", lambda e: canvas.yview_scroll(-1 * int(e.delta / 120), "units")
)

```

Create a frame inside the canvas to contain other frames(containers)

```

frame = tk.Frame(canvas, bg="#FBEEC1")
canvas.create_window((0, 0), window=frame, anchor="nw")

label1 = tk.Label(
    frame, text="1. Original Image", font=("Times New Roman", 20, "bold"),
    bg="#FBEEC1"
)
label1.pack(side=tk.TOP, anchor="w", padx=20, pady=(10, 2), fill="x")
container1 = tk.Frame(
    frame, width=900, height=600, bg="ffffff", borderwidth=2, relief="groove"
)
container1.pack_propagate(False)
container1.pack(
    side=tk.TOP, padx=10, pady=(2, 20), anchor="w", fill="both", expand=True
)

label2 = tk.Label(
    frame, text="2. Grayscale Image", font=("Times New Roman", 20, "bold"),
    bg="#FBEEC1"
)
label2.pack(side=tk.TOP, anchor="w", padx=20, pady=(2, 2), fill="x")
container2 = tk.Frame(
    frame, width=900, height=600, bg="ffffff", borderwidth=2, relief="groove"
)
container2.pack_propagate(False)
container2.pack(side=tk.TOP, padx=10, pady=(2, 2), anchor="w", fill="both",
expand=True)

label3 = tk.Label(
    frame, text="3. Smoothed Image", font=("Times New Roman", 20, "bold"),
    bg="#FBEEC1"
)
label3.pack(side=tk.TOP, anchor="w", padx=20, pady=(2, 2), fill="x")
container3 = tk.Frame(
    frame, width=900, height=600, bg="ffffff", borderwidth=2, relief="groove"
)
container3.pack_propagate(False)

```

```

container3.pack(side=tk.TOP, padx=10, pady=(2, 2), anchor="w", fill="both",
expand=True)

label4 = tk.Label(
    frame, text="4. Binary Image", font=("Times New Roman", 20, "bold"),
    bg="#FBEEC1"
)
label4.pack(side=tk.TOP, anchor="w", padx=20, pady=(2, 2), fill="x")
container4 = tk.Frame(
    frame, width=900, height=600, bg="#ffffff", borderwidth=2, relief="groove"
)
container4.pack_propagate(False)
container4.pack(side=tk.TOP, padx=10, pady=(2, 2), anchor="w", fill="both",
expand=True)

label5 = tk.Label(
    frame, text="5. Final Image", font=("Times New Roman", 20, "bold"),
    bg="#FBEEC1"
)
label5.pack(side=tk.TOP, anchor="w", padx=20, pady=(2, 2), fill="x")
container5 = tk.Frame(
    frame, width=900, height=600, bg="#ffffff", borderwidth=2, relief="groove"
)
container5.pack_propagate(False)
container5.pack(side=tk.TOP, padx=10, pady=(2, 2), anchor="w", fill="both",
expand=True)

label6 = tk.Label(
    frame, text="6. Edged Image", font=("Times New Roman", 20, "bold"),
    bg="#FBEEC1"
)
label6.pack(side=tk.TOP, anchor="w", padx=20, pady=(2, 2), fill="x")
container6 = tk.Frame(
    frame, width=900, height=600, bg="#ffffff", borderwidth=2, relief="groove"
)
container6.pack_propagate(False)
container6.pack(side=tk.TOP, padx=10, pady=(2, 2), anchor="w", fill="both",
expand=True)

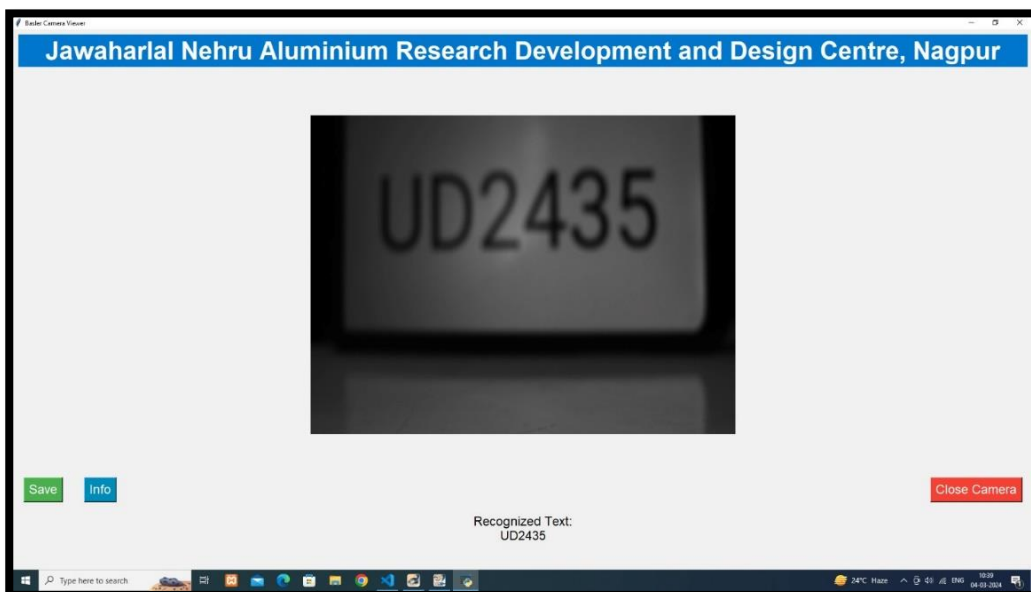
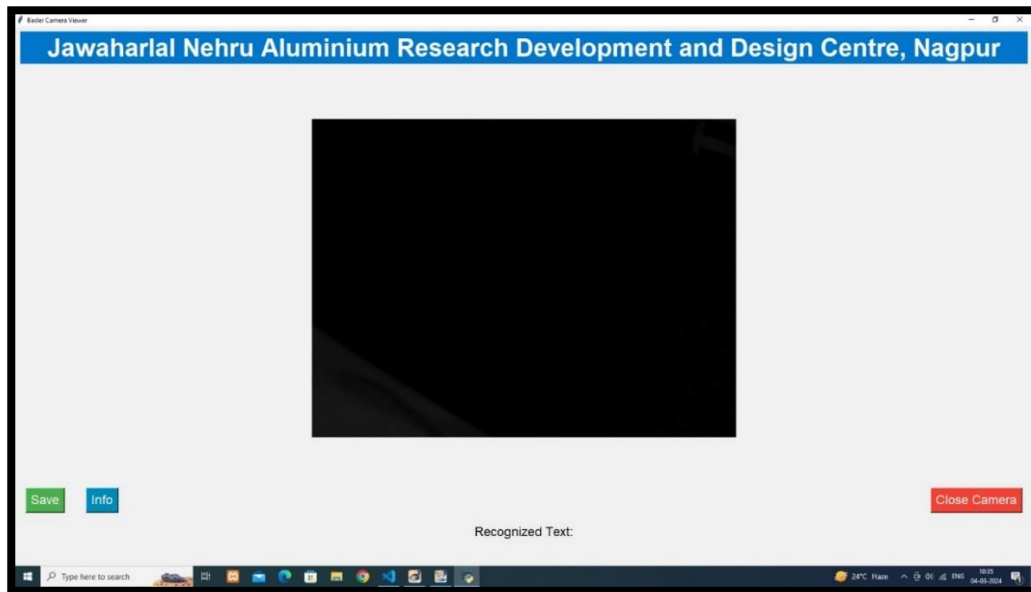
label7 = tk.Label( frame, text="7. Dilated Image", font=("Times New Roman", 20,
"bold"), bg="#FBEEC1"
)
label7.pack(side=tk.TOP, anchor="w", padx=20, pady=(2, 20), fill="x")
container7 = tk.Frame(
    frame, width=900, height=600, bg="#ffffff", borderwidth=2, relief="groove"
)

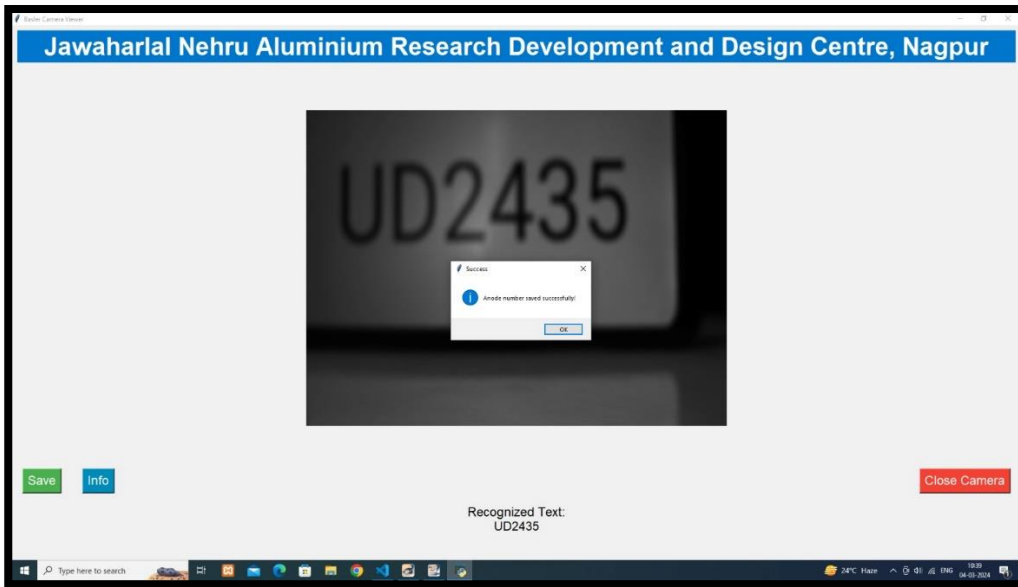
```

```
container7.pack_propagate(False)
container7.pack(
    side=tk.TOP, padx=10, pady=(2, 20), anchor="w", fill="both", expand=True
)

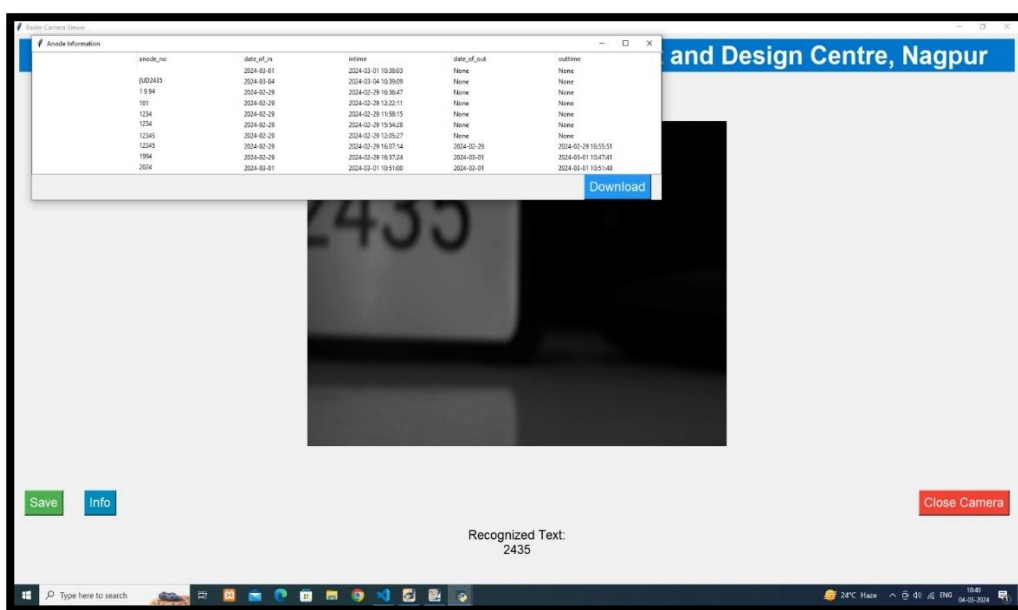
root.mainloop()
```

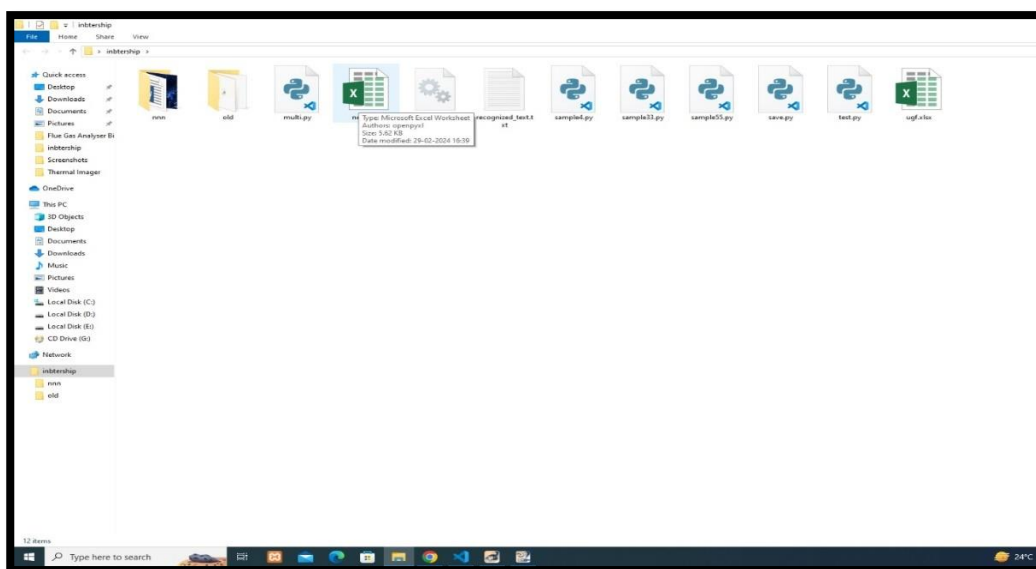
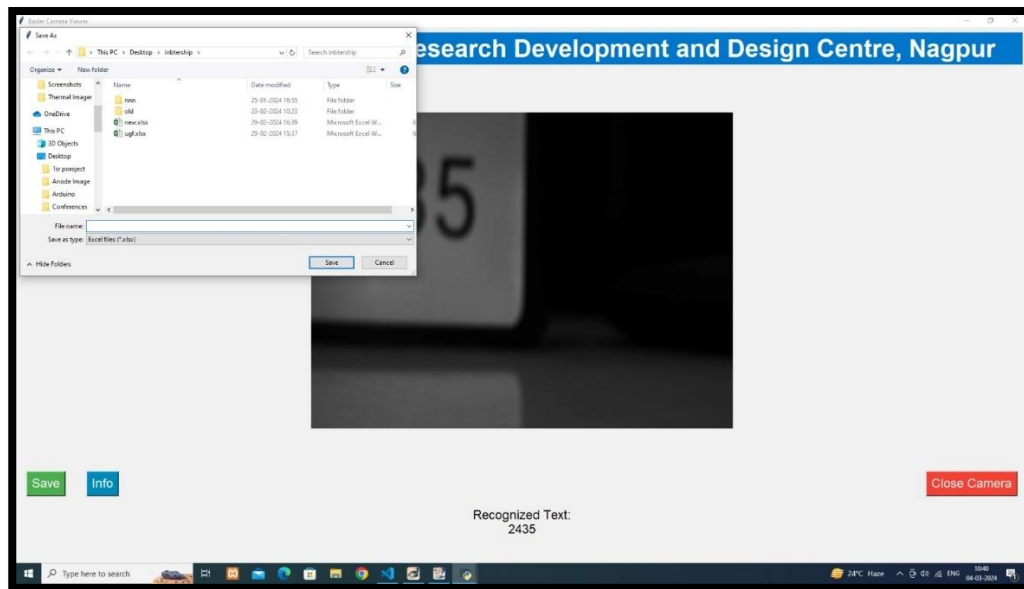
11.2 Steps & Outputs:





anode_no	date_of_in	intime	date_of_out	outtime
UD2435	2024-03-01	2024-03-01 10:38:03	NULL	NULL
1994	2024-03-04	2024-03-04 10:39:09	NULL	NULL
1994	2024-02-29	2024-02-29 16:36:47	NULL	NULL
101	2024-02-29	2024-02-29 12:22:11	NULL	NULL
1234	2024-02-29	2024-02-29 11:58:15	NULL	NULL
1234	2024-02-29	2024-02-29 15:54:28	NULL	NULL
12345	2024-02-29	2024-02-29 12:05:27	NULL	NULL
12345	2024-02-29	2024-02-29 16:07:14	2024-02-29	2024-02-29 16:55:51
1994	2024-02-29	2024-02-29 16:37:24	2024-03-01	2024-03-01 10:47:41
2024	2024-03-01	2024-03-01 10:51:00	2024-03-01	2024-03-01 10:51:48
3243	2024-02-29	2024-02-29 16:29:10	NULL	NULL
67	2024-02-29	2024-02-29 12:36:46	NULL	NULL
8	2024-02-29	2024-02-29 15:21:47	NULL	NULL
8165678	2024-02-29	2024-02-29 16:58:27	NULL	NULL
8C5670	2024-03-04	2024-03-04 11:15:21	NULL	NULL
AB124	2024-02-29	2024-02-29 15:34:42	2024-02-29	2024-02-29 16:17:22
Cd3	2024-02-29	2024-02-29 16:28:29	NULL	NULL
Cd32	2024-02-29	2024-02-29 16:28:52	NULL	NULL
Cd3243	2024-02-29	2024-02-29 15:36:38	2024-02-29	2024-02-29 16:24:14
Cd37	2024-02-29	2024-02-29 16:29:29	NULL	NULL
ER3467	2024-02-29	2024-02-29 16:18:32	NULL	NULL
et 13	2024-03-01	2024-03-01 10:59:54	NULL	NULL
Info	2024-02-29	2024-02-29 16:27:04	2024-02-29	2024-02-29 16:27:11
Jawaharl...	2024-02-29	2024-02-29 16:25:17	NULL	NULL
JITU	2024-02-29	2024-02-29 16:34:02	2024-02-29	2024-02-29 16:37:56
Save	2024-02-29	2024-02-29 16:26:27	NULL	NULL
* NULL	NULL	NULL	NULL	NULL





Final Output:

The screenshot shows a Microsoft Excel spreadsheet with the following data:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
	anode.no	date_of_in	intime	date_of_out	outtime															
1	1994	2024-02-29	2024-02-29 16:36:47																	
2	101	2024-02-29	2024-02-29 12:22:11																	
3	1234	2024-02-29	2024-02-29 13:56:15																	
4	3234	2024-02-29	2024-02-29 15:54:28																	
5	12345	2024-02-29	2024-02-29 12:05:27																	
6	12345	2024-02-29	2024-02-29 16:07:14	2024-02-29	2024-02-29 16:15:10															
7	1994	2024-02-29	2024-02-29 16:17:24																	
8	3243	2024-02-29	2024-02-29 16:29:10																	
9	67	2024-02-29	2024-02-29 12:36:46																	
10	8	2024-02-29	2024-02-29 15:21:47																	
11	AB124	2024-02-29	2024-02-29 15:34:42	2024-02-29	2024-02-29 16:17:22															
12	C65	2024-02-29	2024-02-29 16:26:29																	
13	C612	2024-02-29	2024-02-29 16:26:52																	
14	C61243	2024-02-29	2024-02-29 15:36:38	2024-02-29	2024-02-29 16:24:14															
15	C617	2024-02-29	2024-02-29 16:29:29																	
16	SR2467	2024-02-29	2024-02-29 16:18:32																	
17	Info	2024-02-29	2024-02-29 16:27:04	2024-02-29	2024-02-29 16:27:11															
18	Jasrahatal Ne	2024-02-29	2024-02-29 16:25:17																	
19	JTU	2024-02-29	2024-02-29 16:34:02	2024-02-29	2024-02-29 16:37:56															
20	Save	2024-02-29	2024-02-29 16:26:27																	

11.3 Anode Edge Detection System [Source code]

```
from pypylon import pylon
from PIL import Image, ImageTk
import tkinter as tk
from tkinter import ttk
import easyocr
import cv2
import mysql.connector
from datetime import datetime
from tkinter import messagebox
from tkinter import filedialog
import pandas as pd
```

```
tree = None
```

Database connection parameters

```
db_params = {
    'host': 'localhost',
    'user': 'root',
    'password': 'jnarddc',
    'database': 'jnrddc_1',
    'auth_plugin': 'mysql_native_password'
}
```

```
def open_basler_camera():
    try:
        # Get the list of available devices
        devices = pylon.TlFactory.GetInstance().EnumerateDevices()

        if not devices:
            print("No Basler Camera device found.")
            return
```

Use the first device (you can modify this based on your setup)

```
Camerapylon.InstantCamera(pylon.TlFactory.GetInstance().CreateDevice(devices[0]))
```

Open the camera

```
camera.Open()
```

Check if the camera is open

```
if camera.IsOpen():
```

```
    print("Basler Camera opened successfully.")
```

```
    # Set camera properties for better clarity
```

```
    camera.Width.SetValue(1920) # Set the width of the image
```

```
    camera.Height.SetValue(1080) # Set the height of the image
```

```
    # Get available pixel formats
```

```
    pixel_formats = camera.PixelFormat.GetSymbolics()
```

```
    if not pixel_formats:
```

```
        print("No available pixel formats.")
```

```
        return
```

```
    print("Available Pixel Formats:", pixel_formats)
```

```
    # Set pixel format to RGB8 if available
```

```
    chosen_pixel_format = "RGB8" if "RGB8" in pixel_formats
```

```
else pixel_formats[0]
```

```
    camera.PixelFormat.SetValue(chosen_pixel_format)
```

Create a simple Tkinter window

```
root = tk.Tk()
```

```
root.title("Basler Camera Viewer")
```

```
    # Set the dimensions of the window (width x height)
```

```
    root.geometry("1000x700") # Set your desired width and height
```

```
    # Set a background color
```

```
    root.configure(bg='#f0f0f0')
```

Create a label for the institution name with an increased font size

```
    institution_label = tk.Label(root, text="Jawaharlal Nehru Aluminium Research  
Development and
```

```
Design Centre, Nagpur", font=("Arial", 35, "bold"), fg='white', bg='#0077cc')
```

```
    institution_label.pack(fill="x", pady=10, padx=10)
```

```
    # Create a label to display the camera feed
```

```
    camera_label = tk.Label(root)
```

```
    camera_label.pack(expand=True)
```

Create a label to display the recognized text with an increased font

size

```
text_label = tk.Label(root, text="Recognized Text: ", font=('Helvetica', 18),  
bg='#f0f0f0')  
text_label.pack(side="bottom", pady=20)
```

```
# Initialize EasyOCR reader  
reader = easyocr.Reader(['en'])
```

Variable to store recognized text

```
recognized_text = ""
```

Function to update the camera feed

```
def update_camera_feed():  
    nonlocal recognized_text  
    grab_result=camera.RetrieveResult(5000,  
pylon.TimeoutHandling_ThrowException)  
    if grab_result.GrabSucceeded:  
        frame = grab_result.Array
```

Resize the frame to improve text recognition

```
resized_frame = cv2.resize(frame, (800, 600))  
  
# Convert the frame to a Tkinter-compatible image  
img = Image.fromarray(resized_frame)  
img = ImageTk.PhotoImage(image=img)  
camera_label.imgtk = img  
camera_label.configure(image=img)
```

Perform text recognition on the resized frame

```
results = reader.readtext(resized_frame)  
recognized_text = ""  
for detection in results:  
    recognized_text += f"{detection[1]}\n"  
# Update the text label  
text_label.config(text=f"Recognized Text:\n{recognized_text}")  
root.after(10, update_camera_feed) # Update every 10 millisecon
```

Start grabbing images and update the camera feed

```
camera.StartGrabbing(pylon.GrabStrategy_LatestImageOnly)
```

```
update_camera_feed()
```

Add a save button to save recognized text to a file and database (shifted to the left side)

```
save_button = tk.Button(root, text="Save", font=('Helvetica', 18),  
command=lambda: save_to_database(recognized_text), bg='#4CAF50', fg='white')  
save_button.pack(side="left", padx=20)
```

Add an info button to display camera information (shifted to the left side)

```
info_button = tk.Button(root, text="Info", font=('Helvetica', 18),  
command=lambda:  
display_info(root), bg='#008CBA', fg='white')  
info_button.pack(side="left", padx=20)
```

Add a close button to stop grabbing and close the camera (shifted to the right side)

```
close_button = tk.Button(root, text="Close Camera", font=('Helvetica', 18),  
command=root.destroy, bg='#f44336', fg='white')  
close_button.pack(side="right", padx=20)
```

```
# Update Tkinter main loop  
root.mainloop()
```

```
else:  
    print("Failed to open Basler Camera.")
```

```
except Exception as e:  
    print(f"Error opening Basler Camera: {e}")
```

```
finally:
```

Close the camera in the finally block to ensure it's closed properly

```
if 'camera' in locals() and camera.IsOpen():  
    camera.Close()  
    print("Basler Camera closed.")
```

```
def display_info(parent):  
    connection = None
```

Initialize the connection variable

```
try:  
    connection = mysql.connector.connect(**db_params)  
    cursor = connection.cursor()
```

Fetch the column names dynamically

```
cursor.execute("SHOW COLUMNS FROM stem_analysis")

columns = [column[0] for column in cursor.fetchall()]
```

Display information about all anode numbers in a table form

```
query = f"SELECT * FROM stem_analysis"
cursor.execute(query)
result = cursor.fetchall()
```

```
if result:
```

Create a new window for displaying information in a table form

```
info_window = tk.Toplevel(parent)
info_window.title("Anode Information")

# Create a treeview widget
tree = ttk.Treeview(info_window)

# Define columns dynamically based on the result columns
tree["columns"] = columns

# Format columns
for col in columns:
    tree.column(col, anchor=tk.W, width=200)
    tree.heading(col, text=col, anchor=tk.W)
```

Insert data into the tree view

```
for row in result:
    tree.insert("", tk.END, values=row)

# Pack the treeview
tree.pack(expand=True, fill="both")

# Add a download button to save data to an Excel file (shifted to the left side)
download_button = tk.Button(info_window, text="Download",
font=('Helvetica', 18),
command=lambda: download_data(result, columns), bg='#2196F3', fg='white')
download_button.pack(side="right", padx=20)
```

```

else:
    print("No anode information found.")

except Exception as e:
    print(f"Error retrieving information from the database: {e}")

finally:
    if connection and connection.is_connected():
        cursor.close()
        connection.close()

def display_info_after_update(updated_result):
    if updated_result:
        # Get the existing treeview widget and delete all existing items
        tree.delete(*tree.get_children())

```

Insert data into the tree view with the updated result

```

for row in updated_result:
    tree.insert("", tk.END, values=row)

def save_to_database(text):
    connection = None # Initialize the connection variable
    try:
        connection = mysql.connector.connect(**db_params)
        cursor = connection.cursor()

        # Check if the recognized text already exists in the database
        query_select = "SELECT * FROM stem_analysis WHERE anode_no = %s"
        cursor.execute(query_select, (text,))
        existing_record = cursor.fetchone()

        if existing_record:

```

If the record exists, update date_of_out and outtime

```

current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    query_update = "UPDATE stem_analysis SET date_of_out = %s, outtime = %s
WHERE
    anode_no = %s"
    data_update = (current_time, current_time, text)
    cursor.execute(query_update, data_update)

```

```

connection.commit()
print("Anode number updated in the database.")

# Display popup message
messagebox.showinfo("Success", "Anode number updated successfully!")

else:

```

If the record doesn't exist, insert a new record

```

current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
query_insert = "INSERT INTO stem_analysis (anode_no, date_of_in, intime)
VALUES (%s, %s, %s)"
data_insert = (text, current_time, current_time)
cursor.execute(query_insert, data_insert)

connection.commit()
print("Anode number saved to the database.")

# Display popup message
messagebox.showinfo("Success", "Anode number saved successfully!")

```

Retrieve the entire data after the update or insert operation

```

query_fetch_all = "SELECT * FROM stem_analysis"
cursor.execute(query_fetch_all)
updated_result = cursor.fetchall()

```

Call the display_info_after_update function with the updated result

```

display_info_after_update(updated_result)

except Exception as e:
    print(f"Error saving to database: {e}")
    # Display error popup message
    messagebox.showerror("Error", "An error occurred while saving to the
database.")

finally:
    if connection and connection.is_connected():
        cursor.close()
        connection.close()

```

```

def download_data(result, columns):
    connection = None # Initialize the connection variable
    try:

```

```
connection = mysql.connector.connect(**db_params)
cursor = connection.cursor()
```

```
download_data_to_excel(result, columns)
```

```
except Exception as e:
    print(f"Error downloading data: {e}")
```

```
finally:
    if connection and connection.is_connected():
        cursor.close()
        connection.close()
```

```
def download_data_to_excel(result, columns):
    if not result:
        messagebox.showinfo("No Data", "No data to download.")
    return
```

Ask the user for the file location to save the Excel file

```
file_path = filedialog.asksaveasfilename(
    initialdir='C:/Users/User/Desktop/in',
    defaultextension=".xlsx",
    filetypes=[("Excel files", "*.xlsx")]
)
```

```
# Check if the user canceled the file dialog
if not file_path:
    return
```

```
try:
    # Create a DataFrame from the result
    df = pd.DataFrame(result, columns=columns)
```

```
    # Save the DataFrame to an Excel file
    df.to_excel(file_path, index=False)
```

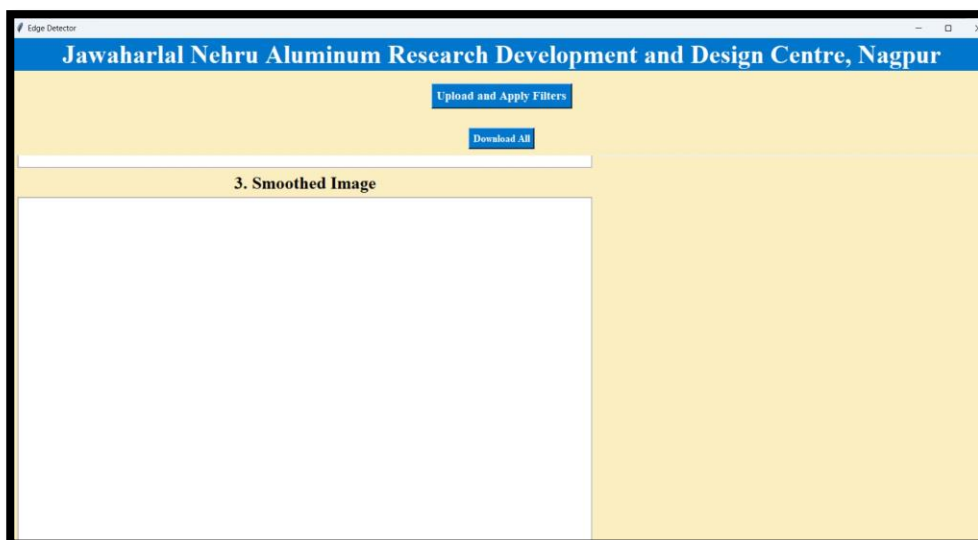
```
    messagebox.showinfo("Success", "Data downloaded successfully!")
```

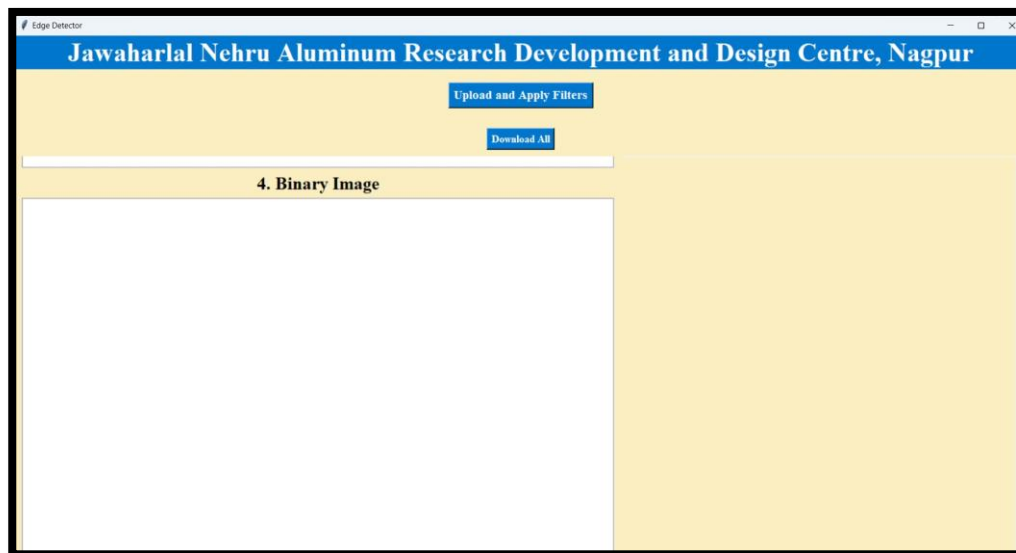
```
except Exception as e:
    print(f"Error downloading data: {e}")
    messagebox.showerror("Error", "An error occurred while downloading data.")
```

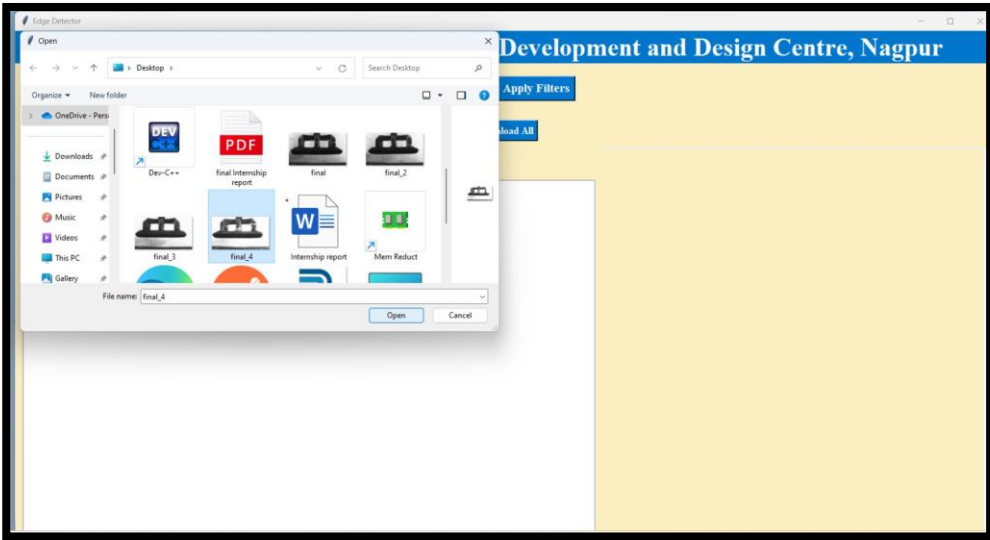
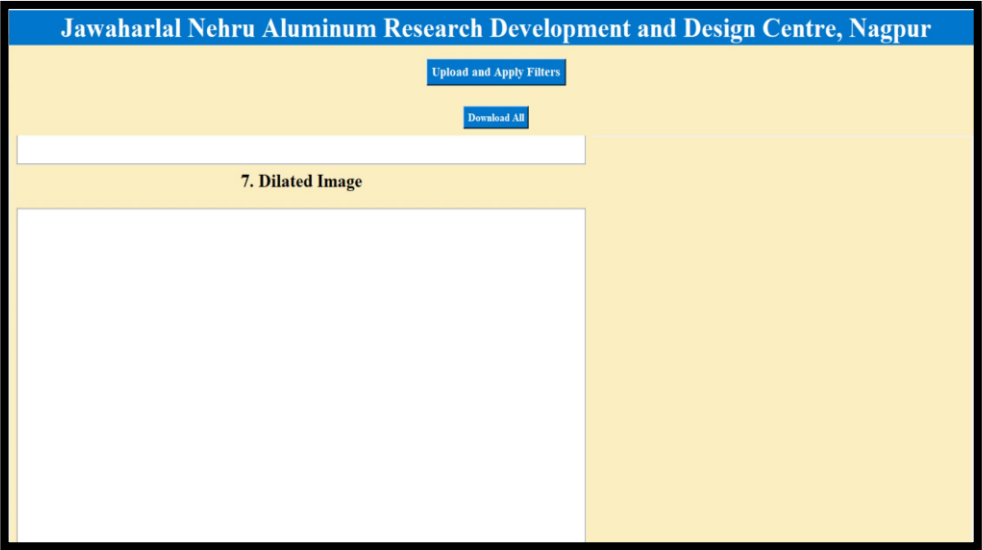
Call the function to open the Basler camera

```
open_basler_camera()
```

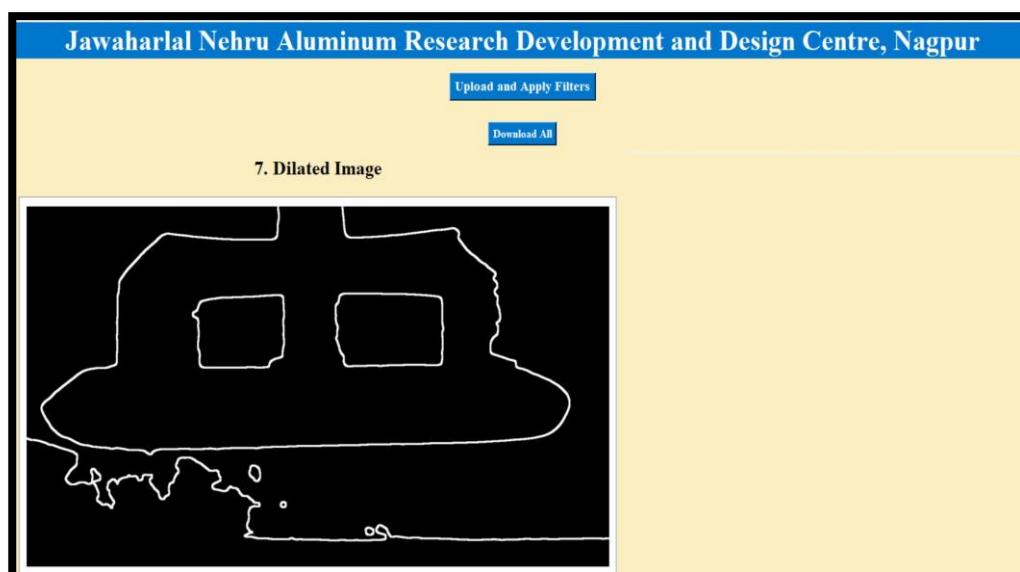
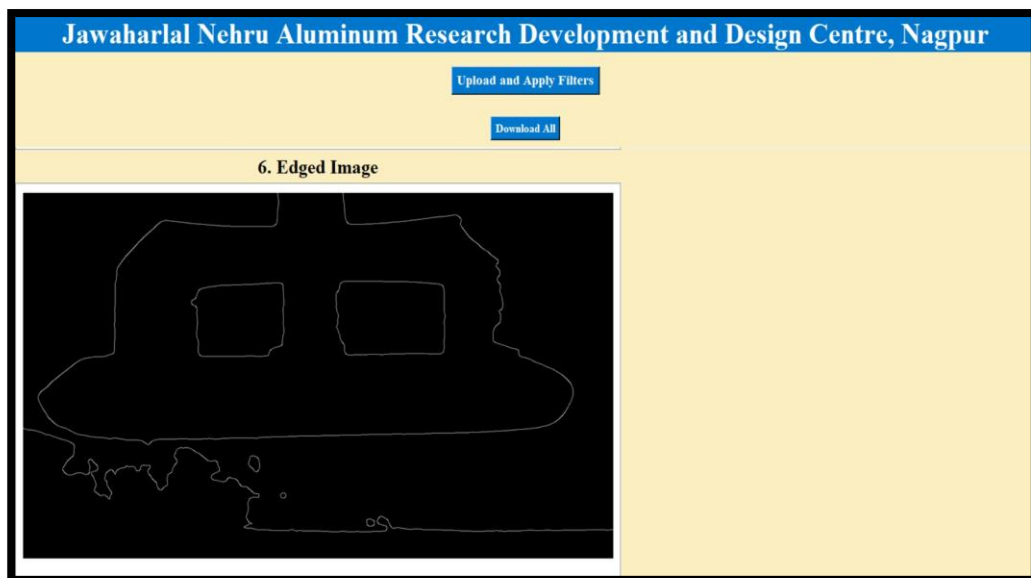
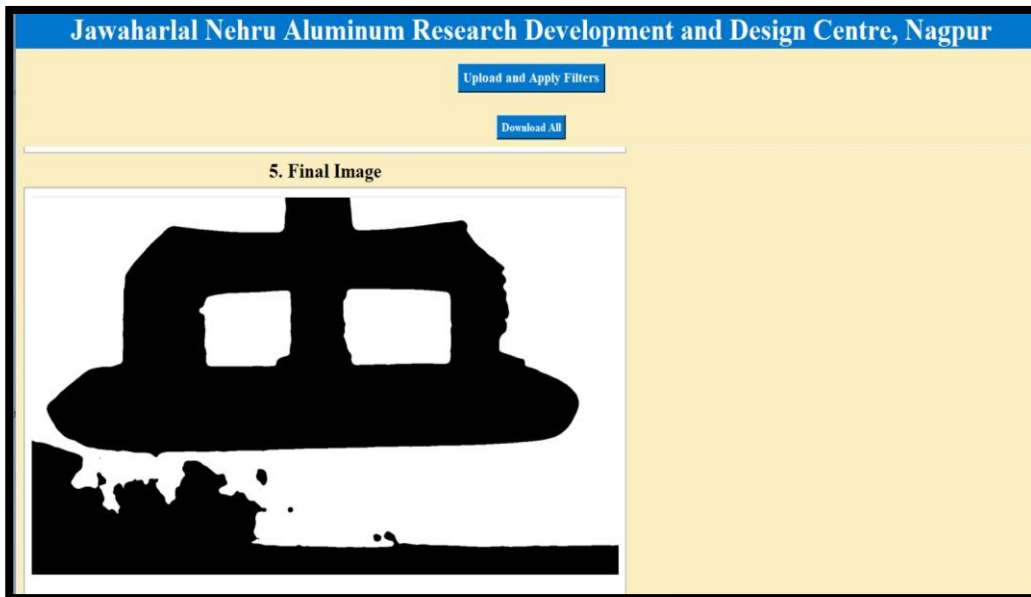
11.4 Steps & Outputs:

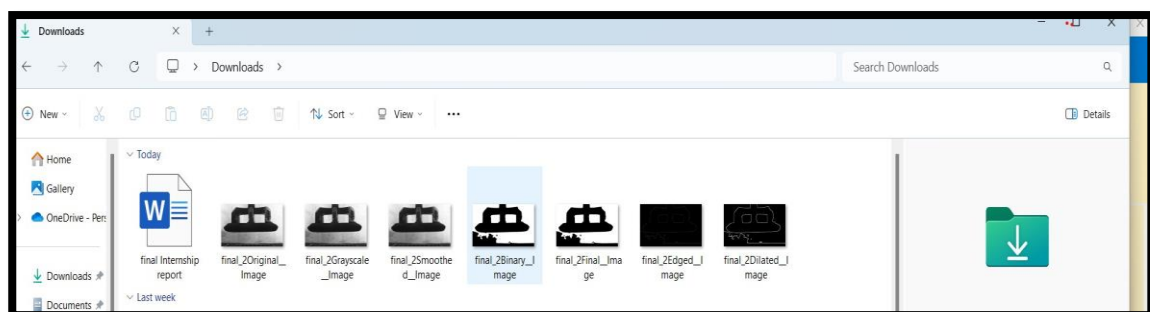
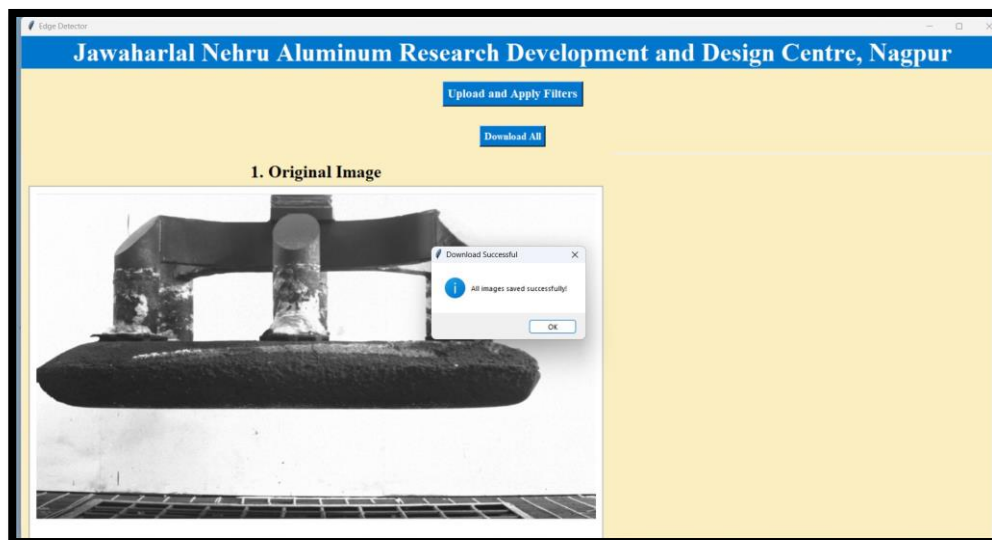












12. Conclusion:

During the internship, our team delved into a myriad of computer vision techniques, ultimately mastering two key methodologies: number detection and edge detection. Through rigorous exploration, we implemented sophisticated approaches such as image preprocessing, contour detection, and character recognition for precise number identification from images. This achievement holds vast implications across domains like optical character recognition (OCR), document processing, and automated data extraction, showcasing our capacity to solve real-world challenges with finesse.

Furthermore, our proficiency extended to edge detection employing advanced algorithms like Sobel, Canny, and Laplacian. This skill set is foundational in pivotal image processing tasks such as object detection, segmentation, and feature extraction. Our adeptness in these techniques not only underscores our competence in computer vision but also positions us to tackle complex problems efficiently and accurately.

Throughout the internship, we cultivated invaluable teamwork skills, collaborating seamlessly to achieve our objectives. This collaborative spirit, coupled with our technical acumen, underscores our potential to excel in diverse projects and contribute meaningfully to the field of computer vision. As we transition from this internship, equipped with practical experience and profound insights, we are poised to embark on more advanced applications such as image classification, object tracking, and scene understanding, paving the way for innovative solutions to future challenges in computer vision.

13. E-References:

1. Website: [OpenCV](<https://opencv.org/>)

- [OpenCV](<https://opencv.org/>)

2. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow" by Aurélien Géron

- You can find this book on platforms like Amazon or the publisher's website.

3. Python Computer Vision by "Joseph Howse

- [Python Computer Vision on Amazon] (<https://www.amazon.com/Python-Computer-Vision-Joseph-Howse/dp/1788474449>)

4. Learning OpenCV 4 with Python" by Joseph Howse, Prateek Joshi, and Michael Beyeler

- (<https://www.amazon.com/Learning-OpenCV-4-Python-artificial/dp/1789344913>)

5. Research Papers

- Research papers can be found on websites like IEEE Xplore, arXiv, or directly on the websites of conferences like [CVPR]
- (<http://cvpr2022.thecvf.com/>),
- [ICCV](<http://iccv2023.thecvf.com/>)
- [ECCV](<https://eccv2024.eu/>)

6. Online Courses and Tutorials

- (<https://www.coursera.org/courses?query=computer%20vision>)
- (<https://www.udemy.com/courses/search/?q=computer%20vision>)

➤ (<https://www.edx.org/learn/computer-vision>)

7. Python for Data Analysis" by Wes McKinney

➤ **[Python for Data Analysis on Amazon]**

➤ (<https://www.amazon.com/Python-Data-Analysis-WranglingIPython/dp/1491957662>)

By exploring these resources, you can gain a solid understanding of number detection, edge detection, and other computer vision applications in Python, and further enhance your skills in this exciting field.